SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0704

COURSE OUTCOME COVERED

CO4:
Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

2. Write a Python function named validate_segment(length_m) to verify whether a given **UTP cable segment** satisfies the IEEE
802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

ANSWERS

1.PYTHON PROGRAMMING FOR STAR TOPOLOGY

1. AIM

To develop a Python program to generate a **Star Topology** for a given number of hosts connected to a central switch and verify all host-to-switch connections.

2. SOFTWARE REQUIRED

- Python 3.x
- Google Colab / Online Python Compiler
- Cisco Packet Tracer

3. EXPLANATION

In a Star Topology, all hosts are connected directly to a **central switch**. The Python program accepts the number of hosts, displays each host-to-switch connection, and specifies the Cat6 UTP cable length.

The graphical topology is implemented in **Cisco Packet Tracer** using one switch and multiple PCs. Each PC is connected individually to the switch using a **Copper Straight-Through cable**.

If one host or its cable fails, the other hosts can continue communication. However, if the central switch fails, communication between all hosts is affected.

CODE:

```
# STAR TOPOLOGY GENERATOR
# Generates:
# 1. ASCII Star Topology
# 2. Graphical Star Topology
# 3. Host-to-Switch connections
# 4. Cat6 UTP cable lengths
# 5. Connection verification

import matplotlib.pyplot as plt
import math
import random

# ------------------------------------------------
# INPUT
# ------------------------------------------------

n = int(input("Enter the number of hosts: "))

if n <= 0:
    print("Please enter a positive number of hosts.")
else:

    switch = "Switch-1"

    # Store host information
    hosts = []

    for i in range(1, n + 1):
```

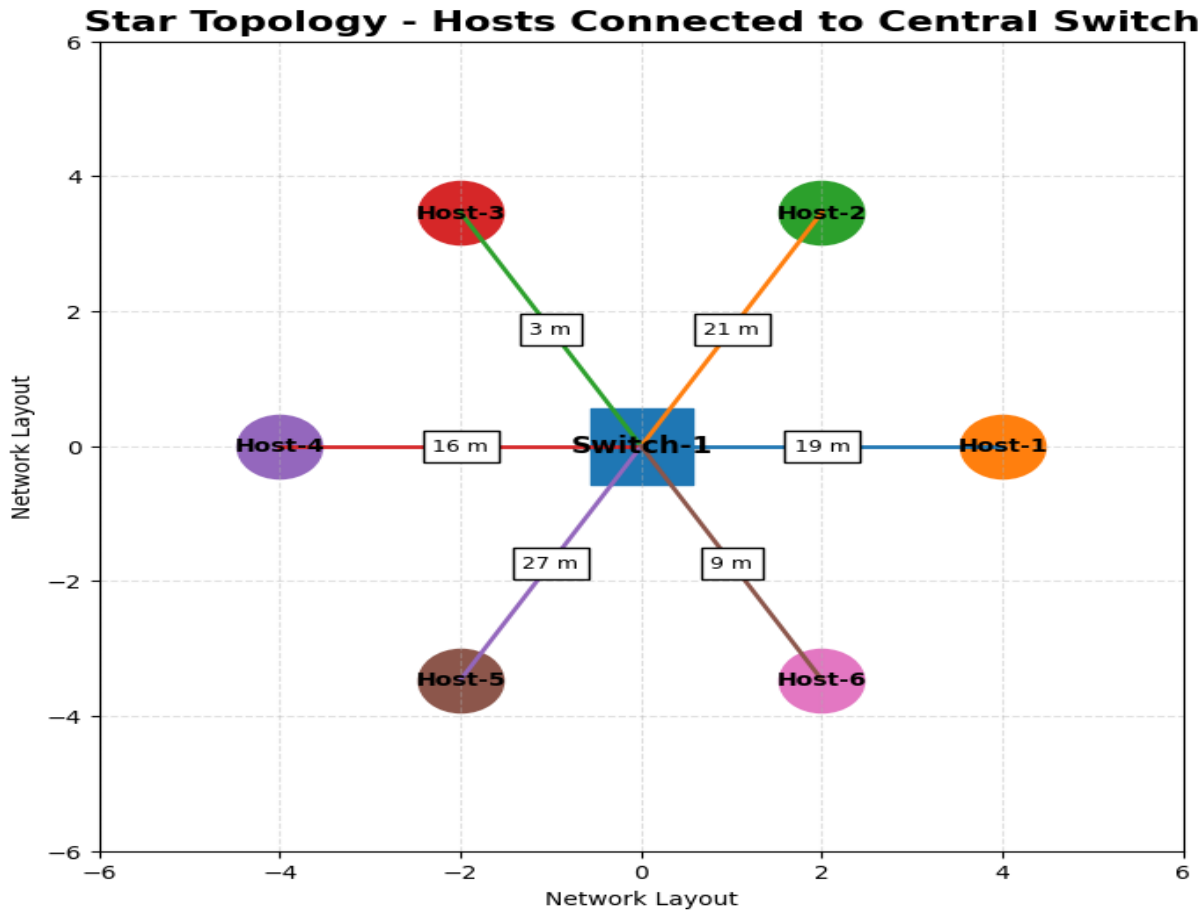
```
    host_name = f"Host-{i}"
    cable_length = random.randint(1, 30)
    hosts.append((host_name, cable_length))

print("\n")
print("=" * 55)
print("            STAR TOPOLOGY")
print("=" * 55)

print()
print("                [Switch-1]")
print("               /    |    \\")

if n == 1:
    print("              [Host-1]")
elif n == 2:
    print("            [Host-1]   [Host-2]")
else:
    # Display first few hosts in ASCII
    for i, (host, length) in enumerate(hosts):
        if i == 0:
            print(f"       [{host}]")
        elif i == n // 2:
            print(f"            \\   |   /")
            print(f"             [{host}]")
        elif i == n - 1:
            print(f"       [{host}]")

print()
print("-" * 55)
print("Host-to-Switch Connections")
print("-" * 55)

for host, length in hosts:
    print(
        f"{host:<10} ---> {switch:<10} "
        f"Cat6 UTP Cable = {length} m"
    )

print("\n")
print("=" * 55)
print("           CONNECTION VERIFICATION")
print("=" * 55)

all_connected = True

for host, length in hosts:

    if length > 0:
        print(
            f"✓ {host} is connected to {switch} "
            f"using {length} m Cat6 UTP cable."
        )
```

```
    else:
        print(f"✗ {host} is NOT connected.")
        all_connected = False

print()

if all_connected:
    print("RESULT: ALL HOSTS ARE CONNECTED SUCCESSFULLY.")
    print("The star topology is valid.")
else:
    print("RESULT: SOME HOSTS ARE NOT CONNECTED.")


plt.figure(figsize=(10, 8))

# Position of central switch
switch_x = 0
switch_y = 0

# Draw switch
plt.scatter(
    switch_x,
    switch_y,
    s=1800,
    marker="s"
)

plt.text(
    switch_x,
    switch_y,
    switch,
    ha="center",
    va="center",
    fontsize=12,
    fontweight="bold"
)

# Radius for hosts
radius = 4

# Draw hosts around switch
for i, (host, length) in enumerate(hosts):

    angle = 2 * math.pi * i / n

    host_x = radius * math.cos(angle)
    host_y = radius * math.sin(angle)

    # Draw cable
    plt.plot(
        [switch_x, host_x],
        [switch_y, host_y],
        linewidth=2
```

```
    )

    # Calculate midpoint for cable label
    mid_x = (switch_x + host_x) / 2
    mid_y = (switch_y + host_y) / 2

    # Display cable length
    plt.text(
        mid_x,
        mid_y,
        f"{length} m",
        fontsize=9,
        ha="center",
        va="center",
        bbox=dict(
            facecolor="white",
            edgecolor="black"
        )
    )

    # Draw host
    plt.scatter(
        host_x,
        host_y,
        s=1200,
        marker="o"
    )

    plt.text(
        host_x,
        host_y,
        host,
        ha="center",
        va="center",
        fontsize=10,
        fontweight="bold"
    )


plt.title(
    "Star Topology - Hosts Connected to Central Switch",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Network Layout")
plt.ylabel("Network Layout")

plt.xlim(-6, 6)
plt.ylim(-6, 6)

plt.grid(True, linestyle="--", alpha=0.4)

plt.gca().set_aspect("equal")
```

plt.show()

4. GRAPHICAL VISUALIZATION OUTPUT



CONNECTIONS:

PC1 → Switch Fa0/1
PC2 → Switch Fa0/2
PC3 → Switch Fa0/3
PC4 → Switch Fa0/4
PC5 → Switch Fa0/5

5. RESULT

Thus, the **Star Topology was successfully implemented** using Python and Cisco Packet Tracer. All hosts were connected to the central switch, and the host-to-switch connections were successfully verified.

2. UTP CABLE LENGTH VALIDATION

1. AIM
To write a Python function validate_segment(length_m) to verify whether a UTP cable segment satisfies the maximum **100-meter Ethernet cable length**.

2. SOFTWARE REQUIRED
- Python 3.x
- Google Colab / Online Python Compiler
-

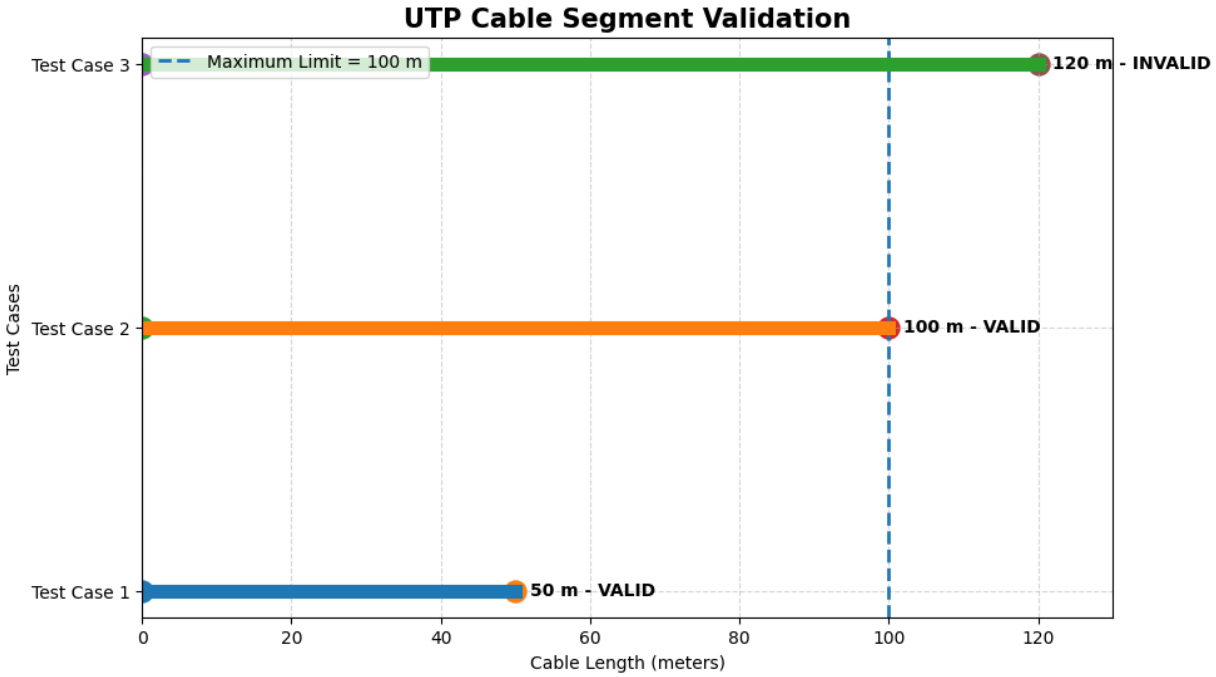
3. EXPLANATION
The function validate_segment(length_m) checks whether the given UTP cable length is within the permissible **100-meter limit**.
- Length ≤ 100 m → **Valid (True)**
- Length > 100 m → **Invalid (False)** with an error message.

CODE:

```
import matplotlib.pyplot as plt


# Function to validate cable length
def validate_segment(length_m):
    if length_m <= 100:
        return True, "VALID"
    else:
        return False, "INVALID"


# Test cases
test_cases = [50, 100, 120]

# Print results
print("========== UTP CABLE VALIDATION ==========")

for length in test_cases:
    result, status = validate_segment(length)

    print(f"Cable Length: {length} m")
    print(f"Result: {result}")
    print(f"Status: {status}")

    if not result:
        print("Error: Cable length exceeds 100 meters.")

    print("-" * 40)

plt.figure(figsize=(10, 6))

# Draw maximum limit line
plt.axvline(
    x=100,
    linestyle="--",
    linewidth=2,
```

```
    label="Maximum Limit = 100 m"
)

# Draw cable segments
for i, length in enumerate(test_cases):

    result, status = validate_segment(length)

    # Cable line
    plt.plot(
        [0, length],
        [i, i],
        linewidth=8
    )

    # Start point
    plt.scatter(0, i, s=150)

    # End point
    plt.scatter(length, i, s=150)

    # Cable length label
    plt.text(
        length + 2,
        i,
        f"{length} m - {status}",
        va="center",
        fontsize=10,
        fontweight="bold"
    )

# Labels
plt.yticks(
    range(len(test_cases)),
    ["Test Case 1", "Test Case 2", "Test Case 3"]
)

plt.xlabel("Cable Length (meters)")
plt.ylabel("Test Cases")

plt.title(
    "UTP Cable Segment Validation",
    fontsize=15,
    fontweight="bold"
)

plt.xlim(0, 130)
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend()
plt.show()
```

4. GRAPHICAL VISUALIZATION OUTPUT



OUTPUT:

========== UTP CABLE VALIDATION ==========

Cable Length: 50 m
Result: True
Status: VALID

Cable Length: 100 m
Result: True
Status: VALID

Cable Length: 120 m
Result: False
Status: INVALID
Error: Cable length exceeds 100 meters.

RESULT:
Thus, the validate_segment(length_m) function was successfully implemented and tested with three different UTP cable lengths. The graphical visualization clearly shows that **50 m and 100 m cables are valid**, while the **120 m cable is invalid** because it exceeds the maximum permissible length of **100 meters**.